SMART CAMPUS EVENT & RESOURCE BOOKING SYSTEM

DBMS Mini Project (UE23CS351A) — Complete Setup, Execution & Live Demo Manual

Institution: PES University, Bengaluru	**Course:** UE23CS351A (DBMS)
Students: Angelo (PES2UG24CS062) & Aditya (PES2UG24CS031)	**Guide:** Dr. Shilpa S, Asst. Professor
Architecture: Hybrid (MySQL 8.0 ACID + MongoDB Vector Search)	**Front-End:** Next.js 14 App Router + Tailwind

1. Prerequisites & What to Install

This project is optimized for a hybrid Windows + WSL 2 developer setup:

Software Component	Where Installed	Purpose in Project	Status / Verification
Node.js & npm (v22.x)	WSL 2 (Ubuntu Linux)	Executes Next.js 14 web server and React components	**Installed** (node -v)
MySQL Server 8.0	Windows (Port 3306)	ACID relational store (7 normalized tables, row locks)	**Installed** (Service: MySQL80)
MongoDB (Atlas / Local)	Cloud Atlas or WSL	768-dim Vector Search & semi-structured telemetry logs	**Optional** (Auto-fallback to SQL)
VS Code + WSL Extension	Windows	Integrated editor for viewing tables, code & .env.local	**Ready**

2. Step-by-Step Execution Guide

Follow these 4 straightforward steps to start the application with live MySQL integration:

Step 2.1 — Initialize MySQL Schema & Seed Data (on Windows)

In your Windows MySQL Command Line Client or Command Prompt, run:

Step 2.2 — Grant WSL Remote Access to Windows MySQL

Because WSL 2 connects across the virtual network bridge, run this in MySQL on Windows:

Step 2.3 — Verify .env.local in Your Project Folder

In VS Code, ensure **.env.local** has your Windows Host IP and exact matching password (no quotes):

```
MYSQL_HOST=172.17.224.1
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=YOUR_REAL_PASSWORD
MYSQL_DATABASE=smart_campus
MONGODB_URI=mongodb://localhost:27017/smart_campus_nosql
NEXTAUTH_SECRET=smart_campus_pes_secret_key_2026
```

Step 2.4 — Launch Next.js Web Server (in WSL Terminal)

Run the development server inside your Ubuntu terminal:

3. Live Evaluation Demo Script (Component by Component)

Use this structured script during your evaluation/presentation. It guides you on what page to open, what feature to click, what database query runs under the hood, and what technical concepts to highlight to the examiners.

Component 1: Project Overview & Landing Page (URL: http://localhost:3000)

What to Show: Open the browser at localhost:3000. Point out the architectural badges, team member names (Angelo & Aditya), and the technical breakdown card.

What to Say: *'Our project solves real-world campus double-booking problems. Standard CRUD systems treat rows independently; our platform uses a Hybrid Polyglot architecture combining MySQL 8.0 for ACID transactions and MongoDB for vector search.'*

Key Concept to Highlight: Polyglot persistence — separating high-concurrency relational locking from unstructured semantic search.

Component 2: Live Operational Dashboard (URL: http://localhost:3000/dashboard)

What to Show: Navigate to the Dashboard. Point out the live metric tiles (Total Venues: 4, Total Bookings: 3, Pending Review: 2, Approved Slots: 1) and the Recent Reservations table.

What Runs Under the Hood: Dynamic SQL aggregate queries with table joins:
SELECT COUNT(*) FROM BOOKINGS WHERE status = 'PENDING';
SELECT b.*, v.name, u.full_name FROM BOOKINGS b JOIN VENUES v ... JOIN USERS u ...

Key Concept to Highlight: Referential Integrity across 3 relational entities (BOOKINGS, VENUES, USERS).

Component 3: Natural Language Semantic Venue Discovery (URL: http://localhost:3000/search)

What to Show: In the search bar, type: *'Find me a quiet seminar space with a projector for 40 students'* and click **Vector Search**.

What to Explain: Show how **Seminar Hall B** appears as the Top Match (Score: ~94.6%). Explain that instead of exact keyword matching, the system generates a 768-dimensional dense vector embedding (via OpenAI text-embedding-3-small) and evaluates Cosine Similarity against MongoDB HNSW vector index with pre-filtered capacity bounds.

Fault-Tolerant Fallback: Explain that if MongoDB is offline, the Next.js API route seamlessly falls back to relational SQL hydration without crashing the user interface.

Component 4: Real-Time Atomic Reservation & Anti-Collision Engine (SHOWSTOPPER)

How to Demonstrate Conflict Prevention Live:
1. Click **Reserve Slot** on **Auditorium 1**.
2. Set the requested time slot to: 2026-11-05 10:00:00 to 2026-11-05 13:00:00 (this window is already reserved in seed.sql).
3. Click **Confirm & Lock**.
4. **Result:** The transaction rolls back and displays: **'Collision Prevented: Venue is already booked during this time window.'**

Under the Hood (InnoDB Pessimistic Row Lock):
SELECT booking_id FROM BOOKINGS WHERE venue_id = 1 AND status IN ('APPROVED', 'PENDING') AND (start_time < :end AND end_time > :start) FOR UPDATE;

Key Concept to Highlight: Concurrency control and serialized transactions preventing race conditions and double-booking anomalies.

Component 5: Administrative Approval State Machine & Audit Trail (URL: http://localhost:3000/admin/approvals)

What to Show: Navigate to the Approvals page. Explain that **Dr. CS HOD (DEPT_ADMIN)** reviews pending reservations that have requires_approval = TRUE.
1. Click **Approve & Log** on Booking #2.
2. The booking status changes to APPROVED instantly.
3. Scroll to the bottom table: an immutable record appears in APPROVAL_LOGS with approver identity, action, comments, and UTC timestamp.

Key Concept to Highlight: Role-Based Access Control (RBAC) and Database Triggers (trg_after_booking_status_update) enforcing accountability.

4. Quick Troubleshooting Cheat Sheet

Problem: Access denied for user 'root'@'172.17.x.x' (using password: YES)
Fix: The password in .env.local does not match the password stored in MySQL for 'root'@'%'.
In MySQL run: ALTER USER 'root'@'%' IDENTIFIED BY 'your_pass'; FLUSH PRIVILEGES; and set MYSQL_PASSWORD=your_pass in .env.local (without quotes).

Problem: Port 3000 already in use
Fix: In WSL terminal run: kill $(lsof -t -i:3000) 2>/dev/null then re-run npm run dev.

Problem: Finding the Windows Host IP in WSL
Fix: In WSL run: ip route show | grep default | awk '{print $3}'. This returns your exact host gateway IP (e.g. 172.17.224.1).